

My Web Development Principles

TypeScript / Next.js を用いた開発で、自分が守る設計・実装・品質基準

・要件定義

要件定義とは、「何を作るのか」を人間にも AI にも伝えられる粒度まで明確化する作業。ただし、一度決めて固定するものではなく、開発・検証を繰り返しながら更新する。ケースバイケースではあるものの。最初の段階では、「作る機能」・「目的」・「解決したい課題」だけ考えれば OK。

・ファイル設計

Next.js を用いるので、コンポーネントによってどう分けるかを考える。これは、AI と相談しながら決める方式で OK。UI・サーバー処理・DB アクセス等の責務を考え、それぞれをどのファイル・コンポーネントに配置するか決定する。1 つのファイルや関数に責務を集中させすぎない。

・コード・デバック

Typescript で 1 ファイル or 1 機能ごとに実装する。(1 ファイルの役割が大きくなったら分割しても OK。) AI は使ってもいいが、説明責任が問われる場合には、Claude Code のような AI エージェントで一気にコードを実装することは NG。これにより説明能力が強化される。更に言うと、基本文法の枠組みを超えるコードを参考文献で残すことや関数名や変数名を工夫することで説明能力を強化するのも大事。

・実行テスト

テストしてエラーが発生した場合、まずエラー内容やログを確認して、どこで問題が発生しているのかを特定する。そのうえで自分なりに原因の仮説を立て、公式ドキュメント・参考文献・AI を利用して原因を調査する。AI には単にエラーを修正させだけでなく、原因の特定や修正方法の提案を補助させる。修正案を自分で理解したうえでコードに反映し、再度テストを行って問題が解消されたことを確認する。また、基本文法の範囲を超える実装や重要な修正については、後から説明できるよう参考文献を残しておく。

・品質とセキュリティ

Web アプリケーションの品質を担保するため、以下の項目を確認する。GitHub Actions で自動化できるものについては、Push や Pull Request 時に自動検証を行う。

- ① Lint : ESLint などを用いて、コードの書き方や潜在的な問題を静的にチェックする。
- ② 型チェック : TypeScript の型チェックを行い、型の不整合がないことを確認する。
- ③ ビルドチェック :

Next.js のビルドを実行し、本番環境向けに正常にビルドできることを確認する。

- ④ Unit Test :

関数やコンポーネントなど、小さな単位の処理が想定通りに動作することを確認する。

- ⑤ Integration Test :

API・データベース・複数の処理などを組み合わせた際に、正しく連携して動作することを確認する。CRUDについても、登録・取得・更新・削除が正常に行えるか確認する。

⑥ E2E Test：ログイン→データ登録→編集→削除など、実際のユーザー操作を想定して Web アプリケーション全体が正常に動作することを確認する。

⑦ Visual Regression Test：コードや CSS を変更した際に、以前の画面と比較して意図しないレイアウト崩れや表示の変化が発生していないか確認する。

⑧ Performance Test：Lighthouse などを利用して、ページの表示速度や Web アプリケーションのパフォーマンスを確認する。

⑨ Dependency Check：利用している npm パッケージなどの依存関係を確認し、既知の脆弱性や問題のある依存関係がないかチェックする。

加えて認証・認可、IDOR、XSS、CSRF、SQL Injection、秘密情報、入力値検証、エラー処理について設計・テストの両面から確認する。

・デプロイ：

Vercel で OK。PostgreSQL を使って認証機能や CRUD アプリを作る場合、Supabase や Supabase Auth を用いることで解決できる。以下が Free プランの参考値。

① PostgreSQL に保存できる DB データ量：500MB

② 1 か月に認証を利用するユーザー数：50000 人

③ データベースからインターネットへ出ていったデータ量：5GB

・ドメインについて

(1)ドメインは Vercel とは別に取得する

Cloudflare Registrar、Namecheap、Squarespace Domains など で .com を取得して、DNS を Vercel へ向ける。ドメインは毎年更新費がかかるので、自動更新を ON にしておく。

(2)HTTPS は Vercel に任せて OK

独自ドメインを Vercel に設定すれば、SSL/TLS 証明書を Vercel 側で用意できるので、<https://example.com> にできる。自分で証明書を管理する必要は基本ない。

(3)www あり・なしを統一する

たとえば、example.com を正式 URL にして、www.example.com → example.com にリダイレクトする。両方バラバラに使わない方が管理しやすい。

(4)Supabase Auth を使うなら URL 設定を忘れない

開発中の localhost:3000 だけでなく、本番の <https://example.com> を Supabase Auth の Site URL / Redirect URLs 等に設定する。Google ログインなど OAuth を使う場合も、リダイレクト URI の設定が重要。